Los entresijos internos de GitHup

Presentado por; Anny Sofia Beltrán García

Presentado a; Carlos Andrés Gómez Vasco

Medicina veterinaria

Universidad agraria de Colombia

UNIAGRARIA

Ciencia de datos

2026

CAPÍTULO 10. LOS ENTRESIJOS INTERNOS DE GIT

Introducción

Git es un sistema de control de versiones que permite registrar, organizar y recuperar los cambios realizados en un proyecto. Aunque normalmente se utiliza mediante comandos como **git add, git commit, git push** y **git pull,** internamente funciona mediante una estructura de objetos, referencias y archivos que permiten almacenar y administrar toda la información. Comprender cómo funciona Git internamente permite conocer cómo guarda los archivos, cómo identifica las versiones, cómo organiza las ramas y cómo transfiere información entre repositorios. Este conocimiento también facilita la recuperación de información y la solución de problemas cuando se trabaja con proyectos de mayor complejidad

10.1 Fontanería y porcelana

Git cuenta con dos tipos generales de comandos: los comandos de alto nivel, conocidos como porcelana, y los comandos de bajo nivel, conocidos como fontanería.

Los comandos de porcelana son los que se utilizan habitualmente para trabajar con los repositorios. Están diseñados para facilitar las actividades del usuario y permitir que las operaciones sean más sencillas

- **Algunos ejemplos son:**
- **git add**: agrega archivos al área de preparación
- **git commit:** registra los cambios realizados
- **git status:** muestra el estado del repositorio
- **git log:** permite consultar el historial de confirmaciones
- **git push:** envía los cambios a un repositorio remoto

Por otra parte, los comandos de fontanería permiten acceder a funciones internas de Git y manipular directamente sus objetos y estructuras. Se utilizan principalmente para tareas avanzadas, automatizaciones y análisis del funcionamiento del sistema

- **Algunos ejemplos son:**
- **git cat-file:** permite consultar información de los objetos almacenados
- **git hash-object:** calcula el identificador de un objeto y puede almacenarlo en Git
- **git ls-tree:** muestra el contenido de un objeto de tipo árbol
- **git update-index**: permite modificar información del índice del repositorio

La diferencia principal es que los comandos de porcelana facilitan el trabajo cotidiano, mientras que los de fontanería permiten comprender y controlar con mayor detalle el funcionamiento interno de Git

10.2 Los objetos Git

Git almacena la información de los proyectos mediante objetos que se guardan dentro de la carpeta interna .git/objects. Cada objeto tiene un identificador que permite reconocerlo y recuperarlo

- **Los objetos principales de Git son:**
- **Blob**: almacena el contenido de un archivo, pero no conserva directamente su nombre ni su ubicación dentro de las carpetas
- **Tree:** representa la estructura de directorios y relaciona los nombres de archivos y carpetas con sus respectivos objetos
- **Commit**: registra una versión del proyecto e incluye información como el autor, la fecha, el mensaje y la referencia al árbol correspondiente

- **Tag:** permite identificar una versión o un punto importante del historial, y puede incluir información adicional sobre la etiqueta

Estos objetos se relacionan entre sí para representar el estado de un proyecto en un momento determinado. Por ejemplo, cuando se realiza una confirmación mediante **git commit**, Git registra un objeto de tipo commit que apunta a un objeto tree. Este árbol contiene referencias a los archivos y directorios del proyecto. Para consultar el tipo de un objeto se utiliza el comando **git cat-file -t**, seguido del identificador correspondiente. También se puede utilizar **git cat-file -p** para mostrar su contenido. La organización mediante objetos permite que Git almacene la información de manera eficiente y conserve las diferentes versiones de los archivos

10.3 Referencias Git

Las referencias de Git son elementos que permiten identificar y localizar confirmaciones dentro del historial de un repositorio. En lugar de tener que escribir identificadores completos de las confirmaciones, se pueden utilizar nombres como main, master o nombres de otras ramas.

Entre las referencias más importantes se encuentran:

- Ramas: apuntan a una confirmación y se actualizan cuando se agregan nuevos cambios a la rama
- Etiquetas: identifican puntos específicos del historial y suelen utilizarse para señalar versiones importantes
- HEAD: indica la posición actual de trabajo, normalmente la rama o confirmación que se encuentra seleccionada

Por ejemplo, cuando se utiliza git switch main, Git cambia a la rama principal llamada main, también se puede consultar la referencia actual mediante **git symbolic-ref HEAD**, cuando HEAD apunta simbólicamente a una rama. Las referencias permiten navegar por el historial y administrar las diferentes líneas de desarrollo de un proyecto sin necesidad de modificar directamente los objetos almacenados.

10.4 Archivos empaquetadores

Git utiliza archivos empaquetadores, conocidos como packfiles, para almacenar varios objetos dentro de un mismo archivo y reducir el espacio que ocupa un repositorio.

Cuando un proyecto tiene muchos cambios, Git puede almacenar objetos individuales y posteriormente agruparlos para mejorar el rendimiento y aprovechar mejor el almacenamiento.

- **Los archivos empaquetadores permiten:**
- Reducir el espacio ocupado por los objetos
- Evitar almacenar repetidamente información similar
- Facilitar la transferencia de datos entre repositorios
- Mejorar la eficiencia al consultar y descargar información

Git utiliza mecanismos de compresión y técnicas de almacenamiento que permiten conservar las diferencias entre objetos relacionados. Para consultar información sobre los objetos almacenados y los archivos empaquetadores se puede utilizar el comando **git count-objects -v**, que muestra datos relacionados con el almacenamiento interno del repositorio.

Los archivos empaquetadores son importantes porque permiten que Git administre proyectos con grandes cantidades de archivos y confirmaciones de manera eficiente.

10.5 Las especificaciones para hacer referencia a los repositorios

Las especificaciones de referencia, conocidas como refspec, permiten indicar qué referencias deben transferirse entre un repositorio local y uno remoto. Se utilizan principalmente en operaciones como git fetch y git push, ya que permiten definir qué ramas o referencias se deben descargar o enviar. Una especificación de referencia puede relacionar una referencia local con una referencia remota.

Entre sus funciones se encuentran:

- Definir qué ramas se descargan desde un repositorio remoto
- Determinar qué referencias se envían al servidor
- Establecer relaciones entre ramas locales y remotas
- Controlar la sincronización de diferentes referencias

10.6 Protocolos de transferencia

Git utiliza diferentes protocolos para intercambiar información entre repositorios. Estos protocolos permiten clonar proyectos, descargar actualizaciones y enviar modificaciones a servidores remotos.

- **Los principales mecanismos de comunicación son:**
- Local: permite trabajar con repositorios ubicados en el mismo computador o en una unidad de almacenamiento compartida
- HTTP y HTTPS: permiten acceder a repositorios mediante direcciones web
- SSH: utiliza una conexión segura para autenticar al usuario y transferir información
- Git: utiliza el protocolo propio de Git para facilitar la comunicación entre repositorios

También existen dos formas de funcionamiento del protocolo HTTP:

- Protocolo tonto: utiliza solicitudes HTTP sencillas para descargar información de un repositorio, sin realizar una negociación avanzada con el servidor
- Protocolo inteligente: permite una comunicación más eficiente entre el cliente y el servidor, negociando qué información necesita transferirse

El protocolo SSH y las conexiones HTTPS son ampliamente utilizados porque permiten establecer comunicaciones seguras con repositorios remotos. La elección del protocolo depende de las necesidades del proyecto, la configuración del servidor y los mecanismos de autenticación disponibles.

10.7 Mantenimiento y recuperación de datos

Git cuenta con herramientas que permiten mantener organizado el repositorio y recuperar información cuando se presentan errores o se pierden referencias a determinadas confirmaciones. El mantenimiento consiste en optimizar el almacenamiento, organizar los objetos y eliminar información que ya no es necesaria. Una de las herramientas utilizadas es git gc, que realiza tareas de limpieza y optimización del repositorio. Por otra parte, la recuperación de datos permite localizar confirmaciones o modificaciones que aparentemente se han perdido.

- **Entre los comandos más importantes se encuentran:**
- **git gc**: realiza tareas de mantenimiento y optimización del repositorio
- **git reflog:** muestra los movimientos recientes de las referencias, como los cambios de posición de HEAD
- **git fsck:** comprueba la integridad de los objetos y permite localizar objetos que no están conectados al historial normal

- **git count-objects -v**: muestra información sobre los objetos almacenados y el espacio que ocupan

El comando git reflog es especialmente útil cuando se necesita recuperar una confirmación después de cambiar de rama, realizar un reinicio o modificar accidentalmente una referencia. Es importante tener en cuenta que no todos los datos eliminados pueden recuperarse indefinidamente. Git puede eliminar objetos que ya no están referenciados durante las tareas de mantenimiento, por lo que es recomendable actuar rápidamente cuando se pierde información

10.8 Variables de entorno

Las variables de entorno permiten modificar determinados aspectos del funcionamiento de Git sin necesidad de cambiar permanentemente los archivos de configuración.Estas variables pueden utilizarse para indicar rutas, establecer información del usuario, seleccionar herramientas y controlar diferentes operaciones del sistema.

- **Algunos ejemplos son:**
- **GIT_DIR**: permite indicar la ubicación del directorio interno del repositorio
- **GIT_WORK_TREE:** permite establecer la ubicación del árbol de trabajo
- **GIT_AUTHOR_NAME**: define el nombre del autor de una confirmación
- **GIT_AUTHOR_EMAIL**: establece el correo electrónico del autor
- **GIT_COMMITTER_NAME**: indica el nombre de quien realiza la confirmación
- **GIT_COMMITTER_EMAIL**: establece el correo electrónico de quien realiza la confirmación
- **GIT_PAGER:** permite seleccionar el programa utilizado para mostrar resultados extensos en la terminal Las variables de entorno son útiles para automatizar procesos,

ejecutar comandos con configuraciones específicas y adaptar Git a diferentes entornos de trabajo.sin embargo, deben utilizarse correctamente, ya que algunas pueden modificar el comportamiento de las operaciones y afectar la forma en que Git identifica los repositorios o registra las confirmaciones

10.9 Ejercicio práctico: explorar el funcionamiento interno de Git

- **Procedimiento:**

1. Crear una carpeta llamada Practica_Internos_Git en el computador
2. Abrir Git Bash e ingresar a la carpeta mediante el comando **cd Practica_Internos_Git**
3. Inicializar un repositorio utilizando **git init**
4. Crear un archivo llamado informacion.txt y escribir un contenido sencillo
5. Agregar el archivo al área de preparación mediante **git add informacion.txt**
6. Crear una confirmación utilizando **git commit -m "Primera confirmación del proyecto"**
7. Consultar el identificador de la última confirmación mediante **git rev-parse HEAD**
8. Identificar el tipo de objeto de la confirmación utilizando **git cat-file -t HEAD**
9. Consultar la información de la confirmación mediante **git cat-file -p HEAD**
10. Mostrar las referencias existentes mediante **git show-ref**
11. Consultar los movimientos recientes de las referencias utilizando **git reflog**
12. Revisar la información de almacenamiento del repositorio mediante **git count-objects -v**

Git también registra objetos, referencias e información que permite reconstruir y consultar el historial de un proyecto

Conclusiones

El funcionamiento interno de Git se basa en una estructura organizada de objetos, referencias y archivos que permite conservar el historial de los proyectos y administrar sus diferentes versiones. Los objetos blob, tree, commit y tag cumplen funciones específicas dentro del almacenamiento de información. Por otra parte, las referencias permiten identificar ramas y confirmaciones, mientras que los archivos empaquetadores ayudan a optimizar el espacio y mejorar la transferencia de datos. Además, los protocolos de comunicación permiten conectar repositorios y compartir modificaciones, mientras que las herramientas de mantenimiento y recuperación facilitan la administración y protección de la información comprender los aspectos internos de Git permite utilizar el sistema de control de versiones de manera más consciente, resolver problemas y aprovechar sus funciones avanzadas en proyectos individuales y colaborativos

Referencias bibliográficas

1. Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2
2. Git. (s. f.). Documentación de Git. https://git-scm.com/docs
3. GitHub. (s. f.). GitHub Docs. https://docs.github.com/es